Diagnostic assisté sur radiographie thoracique

Tâche 3 : automatisation DataOps du pipeline et étude de la fusion image + données cliniques

Binôme [Prénom Nom 1] · [Prénom Nom 2]
Encadrante Aroua Hedhili
Dépôt github.com/mariemtnb/deep-learning-project (tag `task3`)
Date Novembre 2026

Sommaire

1. Rappel du projet et des données
2. Architecture DataOps
   dépôt, étapes, configuration, versionnement, tests, reproductibilité, traçabilité, CI, outils
3. État de l'art de la fusion multimodale
4. Choix argumenté de l'approche
5. Architecture et algorithmes proposés
6. Plan d'intégration et prochaines étapes
Références

1. Rappel du projet et des données

Notre projet vise à repérer des pathologies thoraciques sur une radiographie de face, en s'aidant des quelques informations cliniques qui l'accompagnent. La tâche est multi-label : une même image peut porter plusieurs des 14 pathologies de NIH ChestX-ray14 (épanchement, atélectasie, cardiomégalie, etc.) ou aucune (« No Finding »).

En Tâche 1 nous avions retenu deux sources Kaggle. **NIH ChestX-ray14** [11] est le jeu principal : 112 120 radiographies de 30 805 patients, chacune liée à une ligne d'un fichier CSV qui donne l'âge, le sexe, la vue (PA, debout de dos, ou AP, de face, souvent au lit) et le numéro de suivi du patient. Les deux modalités sont donc une image 1024×1024 en niveaux de gris et un petit vecteur tabulaire de quatre variables. Le jeu **Chest X-Ray Pneumonia** [12] (5 856 images pédiatriques, deux classes) sert de prototype binaire ; il n'a pas de données cliniques et n'entre donc pas dans la fusion.

En Tâche 2 nous avions écrit un script unique (`pipeline.py`) qui nettoyait le CSV, encodait les labels, découpait par patient et produisait un tableau de bord. Il fonctionnait, mais tout y était codé en dur (chemins, seuils, graine), rien n'était testé, et il fallait tout relancer pour la moindre modification. La Tâche 3 reprend ce travail sur deux plans :

- **Partie A** : transformer ce script en un pipeline découpé en étapes, configuré par un fichier YAML, versionné avec DVC, contrôlé par des tests de qualité et exécuté par une CI à chaque push ;
- **Partie B** : étudier les méthodes de fusion multimodale et choisir celle qui convient à nos données, à partir de mesures faites sur ces données et pas seulement de la littérature.

Quelques chiffres issus de l'exécution complète du pipeline reviendront souvent dans la suite :

Mesure	Valeur	Pourquoi elle compte
Images / patients	112 120 / 30 805	3,64 images par patient en moyenne, jusqu'à 184 : le découpage doit se faire par patient.
Images retrouvées sur disque	112 120 (100 %)	Pas de modalité image manquante.
Données cliniques manquantes	0 %	Seuls 16 âges impossibles (148, 411…), réparés.
Images « No Finding »	53,8 %	Classes rares : Hernia 227 images (0,2 %).
Clichés AP	40,0 %	Vue fortement liée à la gravité (section 4).
Découpage train / val / test	77 724 / 8 800 / 25 596	Test officiel NIH conservé, validation tirée par patient.

2. Architecture DataOps

2.1 Organisation du dépôt

Nous avons suivi l'arborescence suggérée par l'énoncé, en ajoutant deux dossiers qui correspondent à des étapes réelles de notre pipeline (`splitting/` et `analysis/`) et un dossier `monitoring/` pour la détection de dérive.

```
configs/          params.yaml (données complètes) · sample.yaml (CI, hérite de params.yaml)
data/raw/nih.dvc  métadonnées NIH, suivies par DVC       data/sample/  échantillon dans Git (0,9 Mo)
data/interim/     tables intermédiaires (DVC)            data/processed/  table finale (DVC)
src/ingestion/    nih_metadata.py, images.py             src/validation/  schemas.py, checks.py
src/preprocessing/ clean.py     src/alignment/ align.py  src/splitting/  split.py
src/analysis/     modalities.py  src/reporting/ …        src/monitoring/  drift.py
src/fusion/       model.py, dataset.py (prototype)       src/training/    réservé Tâche 4
tests/            test_units.py, test_data_quality.py, test_fusion.py
dvc.yaml · dvc.lock · requirements.txt · requirements-dl.txt · Dockerfile · .github/workflows/ci.yml
```

Chaque étape est une fonction `run(cfg)` qui lit ses entrées sur disque, écrit ses sorties sur disque et ne dépend d'aucun état en mémoire. Un point d'entrée unique, `python -m src.cli <etape>`, permet à DVC d'appeler chaque étape séparément et à la CI de tout enchaîner avec `all`.

2.2 Les étapes du pipeline

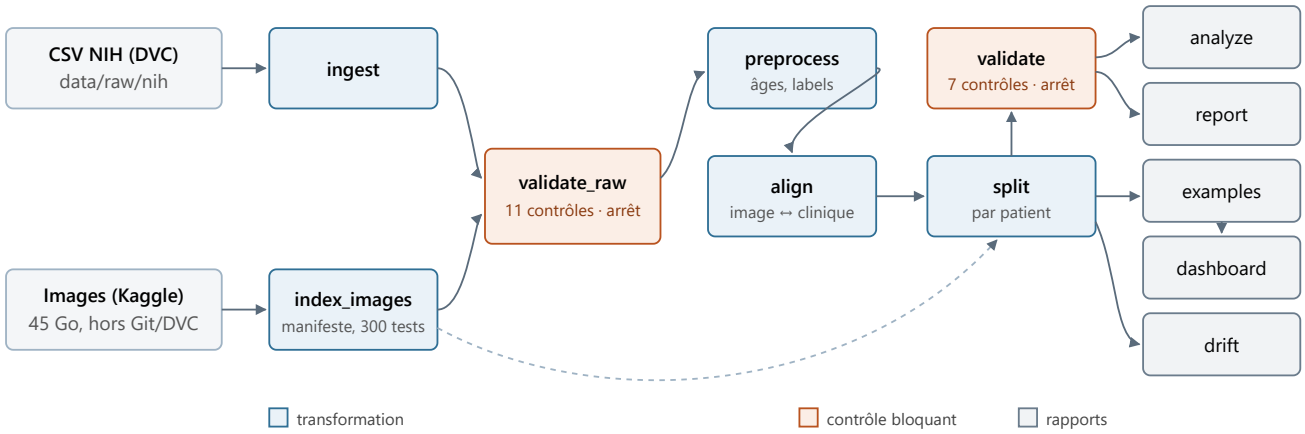


Figure 1 – Graphe des étapes déclarées dans `dvc.yaml` (obtenu aussi par `dvc dag`). En pointillés : l'index des images alimente aussi `align`.

Étape	Ce qu'elle fait	Sortie
`ingest`	Lit le CSV, les boîtes englobantes et les listes officielles. Le CSV a un en-tête cassé (« OriginalImage[Width,Height] » coupé par la virgule, colonne vide finale) : on renomme par position plutôt que par nom.	3 tables Parquet
`index_images`	Liste les fichiers des 12 dossiers (nom, dossier, taille). Ouvre 300 images tirées au hasard pour lire dimensions et mode. Inventorie aussi le jeu pédiatrique.	manifeste, contrôles
`validate_raw`	Contrôles sur les données telles qu'elles arrivent (section 2.5).	JSON de résultats
`preprocess`	Répare les âges hors [1, 100] par la médiane (49 ans) en gardant un indicateur, normalise sexe et vue, éclate les labels en 14 colonnes binaires.	table nettoyée
`align`	Jointure image ↔ ligne clinique sur le nom de fichier ; crée le masque `has_image` / `has_tabular`.	table alignée
`split`	Garde le test officiel, tire 10 % des *patients* du train_val pour la validation.	`dataset.parquet`
`validate`	Fuite de patients, cohérence des labels et des modalités, écarts de prévalence.	JSON de résultats
`analyze`, `report`, `examples`, `dashboard`, `drift`	Mesures pour la Partie B, statistiques, rapport qualité HTML, tableau de bord, comparaison avec la version de référence.	JSON, HTML

Les dossiers d'images ne sont pas déclarés comme dépendances DVC. C'est un choix délibéré : DVC devrait sinon calculer l'empreinte de 45 Go à chaque `dvc status`, ce qui prend plusieurs minutes pour une donnée qui ne change jamais. Leur intégrité est vérifiée autrement, par le manifeste et les contrôles de `validate_raw` : si une image disparaît, la couverture tombe sous 100 % et le pipeline s'arrête.

2.3 Configuration externalisée

Aucun script ne contient de chemin ni de seuil. Tout est dans `configs/params.yaml`, organisé par section : `paths`, `ingest`, `images` (taille attendue, modes autorisés, nombre d'images contrôlées), `preprocess` (bornes d'âge, liste des pathologies), `split`, `quality` (seuils), `drift`, et deux sections réservées, `fusion` et `tracking`. Le fichier `sample.yaml` déclare

`extends: params.yaml` et ne remplace que les chemins et la taille d'image (128×128 dans l'échantillon). Nous n'avons pas utilisé Hydra : un chargeur de 40 lignes avec fusion récursive suffit et évite une dépendance de plus.

Dans `dvc.yaml`, chaque étape déclare les clés de paramètres qu'elle lit. Modifier `split.val_fraction` relance `split` et ce qui en dépend ; modifier `quality.max_invalid_age_rate` ne relance que les deux validations. Nous l'avons vérifié : après un `dvc repro` complet, un second appel répond « Data and pipelines are up to date » sans rien exécuter.

2.4 Versionnement des données

Quoi	Où	Pourquoi
Code, configuration, `dvc.yaml`, `dvc.lock`, rapports JSON	Git	Petits fichiers texte, à relire en revue.
Métadonnées brutes (9,5 Mo), tables intermédiaires et finales	DVC, stockage `shared`	Données, donc hors Git ; versions liées au commit par `dvc.lock`.
Échantillon de 40 patients (0,9 Mo)	Git	La CI doit pouvoir s'exécuter sans accès au stockage.
Images (45 Go + 2,3 Go)	Kaggle, non copiées	Immuables, déjà hébergées publiquement ; les dupliquer dans un cache DVC doublerait l'espace disque.

Le stockage distant est un dossier partagé : sur nos machines, un dossier Google Drive synchronisé par l'application Drive. DVC le voit comme un dossier local. Chaque membre indique son propre chemin avec `dvc remote modify --local shared url …`, écrit dans `.dvc/config.local` qui n'est pas versionné. Nous avons écarté le remote Google Drive natif de DVC : depuis 2024 il demande de créer son propre projet Google Cloud et un compte de service, ce qui est disproportionné pour un binôme. Pour un tiers, la procédure complète est dans le README : `git clone`, réglage du remote, `dvc pull`, puis téléchargement des images depuis Kaggle. Sans accès au dossier partagé, il suffit de copier les quatre fichiers de métadonnées du téléchargement NIH dans `data/raw/nih/` : `dvc repro` refait le reste en une dizaine de secondes.

2.5 Tests de qualité des données

Les contrôles ont deux niveaux. Un contrôle **critique** qui échoue lève une exception, l'étape s'arrête avec le code 1 et DVC n'exécute pas les étapes suivantes. Une **alerte** est écrite dans le rapport sans arrêter le pipeline. Les schémas sont écrits avec pandera ; les contrôles plus métier (fuite, couverture) sont des fonctions pandas courtes, plus lisibles qu'un schéma pour ce genre de règle.

Catégorie	Contrôle	Seuil	Résultat (complet)
Schéma / types	Table clinique brute : 11 colonnes exactes, types, nom d'image `\d{8}_\d{3}.png` unique, sexe ∈ {M, F}, vue ∈ {PA, AP}	0 échec	**OK**
	Images : dimensions 1024×1024 (sur 300 tirées)	≤ 1 % hors format	**0 %**
	Images : mode niveaux de gris (L ou RGBA)	0 autre	**298 L, 2 RGBA**
Manquants, doublons, aberrants	Manquants sur les colonnes clés	0 %	**0 %**
	Doublons de nom d'image	0	**0**
	Âges hors [1, 100] avant réparation	≤ 0,1 %	**0,014 % (16)**
Cohérence inter-modalités	Chaque ligne clinique a son image sur disque	100 %	**100 %**
	Chaque boîte englobante pointe vers une image connue	0 orpheline	**0**
	Masque `has_image` conforme au dossier trouvé ; échantillons complets	100 %	**100 %**
	Images sans ligne clinique (alerte)	0	**0**

Fuite	Listes officielles disjointes et complètes	0	**0**
	Patients présents dans plus d'un ensemble	0	**0**
Labels	« No Finding » exclusif, `n_findings` = somme des labels	0 incohérence	**OK**
	Écart de prévalence val / test (alerte)	≤ 5 points	**7,4 points**

L'alerte de la dernière ligne n'est pas un bug, et c'est sans doute le résultat le plus utile de cette partie. Le test officiel NIH a été construit en réservant des patients entiers, et il se trouve que ce sont des patients beaucoup plus suivis : 20 radios de suivi en moyenne contre 5 en train, 56,6 % de clichés AP contre 34,8 %, et 10,4 % de pneumothorax contre 3,0 %. Notre validation, tirée dans le train_val, ressemble au train et pas au test. Les scores de validation seront donc optimistes, et il faudra le dire quand nous publierons nos résultats.

Nous ne voulions pas des tests qui passent toujours. Les tests `test_data_quality.py` corrompent volontairement une copie des données (patient déplacé d'un ensemble à l'autre, image supprimée, doublon, sexe « X », dimensions fausses, label positif sur une image « No Finding ») et vérifient que le bon contrôle échoue. Le script `scripts/demo_quality_failure.py` fait la même chose de bout en bout pour la démonstration :

```
$ python scripts/demo_quality_failure.py --fault leak
-> 00000147_000.png déplacée de test vers train_val (patient 147)
[validate] FAIL fuite de patients entre train/val/test (valeur=1, seuil=0)
PIPELINE ARRÊTÉ : contrôle(s) critique(s) en échec : ['fuite de patients entre train/val/test']
code de sortie : 1 (arrêt attendu)
```

2.6 Reproductibilité

- **Graine fixée** (`seed: 42`) pour le tirage des patients de validation et des images contrôlées. Le tirage trie d'abord les identifiants : l'ordre de lecture du disque n'a plus d'influence.
- **Dépendances figées** dans `requirements.txt` (versions exactes) ; torch et MLflow à part dans `requirements-dl.txt` pour que la CI des données reste légère.
- **Test explicite** : `test_rerun_is_reproducible` relance l'étape `split` et compare la table produite à la précédente.
- **Docker** (bonus) : l'image `python:3.13-slim` installe les dépendances et exécute le pipeline sur l'échantillon ; les images complètes se montent en volume.

2.7 Observabilité et traçabilité

Chaque étape s'exécute dans un contexte qui chronomètre, journalise et écrit une fiche `reports/stages/<etape>.json` : volumes en entrée et en sortie, nombre de lignes rejetées ou réparées, durée, empreinte MD5 de chaque fichier lu et écrit, commit Git et fichier de configuration. Avec ces fiches, on peut remonter de n'importe quelle table produite aux fichiers sources exacts qui l'ont générée. Les mêmes événements partent en JSON lines dans `logs/pipeline.jsonl`, ce qui permet de les filtrer ou de les charger dans un outil de suivi.

L'étape `report` produit un rapport qualité HTML autonome (bonus) qui reprend tous les contrôles, leurs valeurs, leurs seuils et la durée de chaque étape. La fiche de données (`docs/DATA_CARD.md`) décrit les sources, licences, volumes, prétraitements, biais connus et usages à éviter. Enfin, l'étape `drift` (bonus) résume la version courante (déciles d'âge, sexe, vue, ensembles, prévalence des 14 classes) et la compare à un profil de référence enregistré avec l'indice de stabilité de population (PSI) : en dessous de 0,10 la distribution est stable, au-dessus de 0,25 on parle de dérive. Ce sera utile le jour où nous ajouterons une autre source (MIMIC-CXR par exemple).

2.8 Intégration continue

Le workflow `.github/workflows/ci.yml` tourne sur chaque push et chaque pull request :

1. installation de Python 3.13, de `requirements.txt` et de torch CPU ;

2. `ruff check .` (lint et ordre des imports) ;
3. `dvc dag`, qui échoue si `dvc.yaml` est mal formé ou contient un cycle ;
4. le pipeline complet sur `data/sample` (≈ 3 s) ;
5. `pytest` : 17 tests unitaires, de données et du prototype de fusion ;
6. l'échec contrôlé, qui doit renvoyer le code 1 (sinon la CI échoue : un contrôle qui ne bloque plus est une régression) ;
7. publication des rapports de l'échantillon en artefact du run.

2.9 Choix des outils

Besoin	Choix	Raison
Versionnement des données	DVC	S'appuie sur Git, gère aussi l'orchestration ; LakeFS demande un serveur. Git LFS est déconseillé pour ces volumes.
Orchestration	DVC pipelines	Relance incrémentale native et lien direct avec les versions de données. Airflow ou Prefect seraient surdimensionnés pour un pipeline de 9 s sur une seule machine.
Validation	pandera + pytest	Schémas déclaratifs en Python, rapport d'erreurs exploitable. Great Expectations demande beaucoup de configuration pour peu de règles.
Configuration	YAML + chargeur maison	Héritage simple suffisant, une dépendance de moins que Hydra.
CI	GitHub Actions	Le dépôt est sur GitHub, gratuit pour un dépôt public.
Suivi d'expériences	MLflow (préparé)	Fonctionne en local sans compte, contrairement à W&B.
Format	Parquet	Types conservés, 1,7 Mo contre 14,8 Mo en CSV pour la table finale.

2.10 Préparer la suite

Les étapes `extract_features`, `train` et `evaluate` sont déjà écrites, en commentaire, à la fin de `dvc.yaml`, avec leurs dépendances. La section `tracking` de la configuration et la fonction `start_tracking` de `src/training/train.py` fixent l'URI et le nom de l'expérience MLflow. Le Dataset PyTorch (`src/fusion/dataset.py`) lit directement la table produite par le pipeline, avec son masque de modalités.

3. État de l'art de la fusion multimodale

Baltrušaitis et al. [1] distinguent cinq problèmes en apprentissage multimodal : représentation, traduction, alignement, fusion et co-apprentissage. Nous nous intéressons ici à la fusion, c'est-à-dire à la façon de combiner les modalités pour prédire. La revue de Huang et al. [13], centrée sur l'imagerie médicale et le dossier patient, montre que la question se pose surtout en ces termes : à quel moment du réseau combiner, et avec quel opérateur.

3.1 Les trois niveaux de fusion

Fusion précoce. On combine les modalités avant le modèle principal : soit les données brutes (par exemple en ajoutant l'âge comme canal constant de l'image), soit des caractéristiques extraites séparément puis concaténées et données à un seul classifieur. C'est simple et cela laisse le modèle apprendre toutes les interactions, mais les représentations doivent être compatibles en échelle et en dimension. Coller 4 nombres à 50 000 pixels ne fonctionne pas : le signal clinique est noyé.

Fusion tardive. On entraîne un modèle par modalité et on combine les prédictions : vote, moyenne pondérée, ou un méta-modèle appris (stacking). C'est modulaire, robuste à l'absence d'une modalité (on utilise le modèle restant), et la

contribution de chacune se lit dans les poids. En revanche, aucune interaction fine n'est possible : le modèle image ne peut pas interpréter une silhouette cardiaque différemment selon la vue.

Fusion intermédiaire (ou hybride). Chaque modalité a son encodeur, et les représentations latentes sont combinées à l'intérieur du réseau, entraîné de bout en bout. C'est le cadre le plus utilisé aujourd'hui, et celui où se placent presque tous les mécanismes ci-dessous. Le prix à payer est un entraînement plus délicat (une modalité peut dominer) et une interprétabilité plus faible.

3.2 Mécanismes de fusion

Concaténation, somme, produit. La concaténation $z = [h_A ; h_B]$ suivie d'un MLP est la base de référence. La somme pondérée impose des dimensions égales ; le produit élément par élément ($h_A \odot h_B$) introduit une interaction multiplicative sans augmenter la dimension.

Gated Multimodal Unit (GMU) [4]. Une porte $z = \sigma(W_z[x_A ; x_B])$ décide, pour chaque dimension et chaque exemple, de la part de chaque modalité : $h = z \odot \tanh(W_A x_A) + (1 - z) \odot \tanh(W_B x_B)$. Peu coûteux, et la porte se lit comme une mesure de contribution.

FiLM [5]. Une modalité « conditionne » l'autre : elle produit, pour chaque canal d'une carte de caractéristiques, un gain γ et un biais β, et la carte devient $\gamma \odot F + \beta$. Conçu pour le raisonnement visuel guidé par une question, FiLM convient quand une modalité courte doit changer la façon de lire une modalité riche. Le coût est une couche linéaire.

Tensor Fusion Network (TFN) [2]. On calcule le produit extérieur des représentations, chacune augmentée d'un 1 : $(h_A \oplus 1) \otimes (h_B \oplus 1)$. On obtient toutes les interactions unimodales, bimodales (et trimodales pour trois modalités), mais la dimension explose : $1025 \times 65 \approx 67\,000$ pour nos dimensions. **LMF** [3] factorise le tenseur de poids en rang faible r et ramène le coût à une somme de r produits de projections, avec des résultats proches.

Pooling bilinéaire compact (MCB, MFB) [14, 15]. Même idée d'interactions d'ordre deux, approximées par projection aléatoire (Count Sketch + FFT pour MCB) ou par factorisation (MFB). Très utilisés en question-réponse visuelle, quand les deux modalités sont de grande dimension.

Attention croisée et transformers multimodaux. Les tokens d'une modalité servent de requêtes sur les tokens de l'autre : $\mathrm{softmax}(Q_A K_B^\top / \sqrt{d}) V_B$ [16]. Le Multimodal Transformer [6] le fait dans les deux sens et gère des séquences non alignées. C'est le mécanisme le plus expressif, mais il suppose des séquences de tokens des deux côtés et beaucoup de données ; les paires de modalités coûtent en $O(n \cdot m)$.

Goulots d'attention [7]. Les modalités n'échangent qu'à travers un petit nombre de tokens partagés, ce qui force à condenser l'information et réduit le coût de l'attention croisée complète.

Alignement contrastif (CLIP) [8]. Deux encodeurs projettent les paires dans un espace commun et une perte contrastive rapproche les paires correspondantes. C'est une méthode de pré-entraînement plus que de fusion : elle suppose deux modalités riches et sémantiquement équivalentes (image et texte), avec des centaines de milliers de paires.

Fusion par graphes. Les modalités ou les échantillons deviennent des nœuds, et un réseau de graphe [17] propage l'information. Pertinent quand la relation entre entités est connue (patients similaires, régions anatomiques), plus difficile à mettre en œuvre.

3.3 Difficultés propres au multimodal

- **Hétérogénéité** : une carte 1024×7×7 d'un côté, 4 scalaires de l'autre ; il faut des encodeurs et des projections propres à chaque modalité.
- **Alignement** : temporel (vidéo et audio) ou sémantique (mot et région d'image). Dans notre cas, il est donné par la clé `image`.

- **Modalités manquantes à l'inférence** : la fusion tardive s'en accommode, la fusion intermédiaire demande un jeton « absent » appris ou un entraînement avec suppression aléatoire de modalité (ModDrop [18]).
- **Déséquilibre entre modalités** : Wang et al. [19] montrent qu'un réseau multimodal fait parfois moins bien que le meilleur unimodal, parce que la modalité qui apprend le plus vite domine et que l'autre est sous-entraînée. Des pertes par modalité ou un mélange de gradients corrigent ce biais.
- **Coût de calcul** : TFN et l'attention croisée complète sont quadratiques en dimension ou en nombre de tokens.
- **Interprétabilité** : facile en fusion tardive (poids), possible avec une porte (GMU) ou un conditionnement (FiLM), plus floue avec l'attention profonde.

3.4 Tableau comparatif

Approche	Principe	Avantages	Limites	Conditions d'usage / réf.
Fusion précoce	Concaténer entrées ou caractéristiques avant un modèle unique	Simple ; toutes les interactions sont apprenables	Échelles incompatibles ; la petite modalité est noyée ; pas de modalité manquante	Modalités de même nature et de même granularité [1]
Fusion tardive	Un modèle par modalité, puis vote, moyenne pondérée ou stacking	Modulaire ; robuste à l'absence ; contributions lisibles	Aucune interaction fine entre modalités	Modalités indépendantes, chacune suffisante ; baseline [1, 13]
Intermédiaire – concaténation	$[h_A ; h_B]$ puis MLP, bout en bout	Simple ; interactions via le MLP	Interactions faibles ; domination d'une modalité	Baseline de référence [13]
Intermédiaire – somme / produit pondérés	Combinaison élément par élément de projections	Léger ; interaction multiplicative (produit)	Dimensions égales ; interactions limitées	Modalités de poids comparable
GMU	Porte sigmoïde qui pondère chaque modalité par dimension	Peu coûteux ; contribution mesurable	Reste une combinaison convexe, pas un conditionnement	Modalités d'utilité variable selon l'exemple [4]
FiLM	Une modalité produit γ, β qui modulent les canaux de l'autre	Interaction fine pour un coût d'une couche linéaire ; adapté à un contexte court	Asymétrique ; suppose une modalité « contexte »	Modalité courte qui change l'interprétation d'une modalité riche [5]
TFN / LMF	Produit extérieur des représentations (LMF : rang faible)	Interactions explicites d'ordre 1 à M	Dimension explosive (TFN) ; LMF plus léger	Peu de modalités de petite dimension [2, 3]
Pooling bilinéaire (MCB, MFB)	Interactions d'ordre 2 approchées (sketch, factorisation)	Expressif ; plus léger que le bilinéaire exact	Hyperparamètres sensibles ; peu interprétable	Deux modalités denses de grande dimension (VQA) [14, 15]
Intermédiaire – attention croisée	Tokens d'une modalité en requêtes sur l'autre	Le plus expressif ; gère le non-aligné	Beaucoup de données ; coût $O(nm)$; tokens requis des deux côtés	Séquences riches des deux côtés [6, 16]
Goulots d'attention	Échange via quelques tokens partagés	Moins coûteux que l'attention complète	Architecture transformer complète	Vidéo + audio, grands volumes [7]
Alignement contrastif	Espace commun appris par perte contrastive	Pré-entraînement puissant ; zéro-shot	Paires sémantiquement équivalentes, très nombreuses	Image + texte à grande échelle [8]

Fusion par graphes	Nœuds = modalités ou patients, propagation par GNN	Exploite des relations connues	Construction du graphe ; mise en œuvre lourde	Relations explicites entre entités [17]

4. Choix argumenté de l'approche

Avant de choisir, nous avons mesuré trois choses sur nos données (étape `analyze`, fichier `reports/modality_analysis.json`).

Ce que vaut la clinique seule. Une régression logistique par pathologie, entraînée sur le train avec les quatre variables cliniques, obtient une AUC moyenne de **0,66** sur la validation. Ce n'est pas rien, mais c'est loin des modèles image publiés sur ce jeu (0,74 en moyenne dans l'article d'origine [11], 0,84 pour CheXNet [20], sur un découpage différent). Le détail est plus parlant que la moyenne :

Pathologie	AUC clinique seule	Prévalence AP / PA	Lecture
Hernia	0,86	0,28	Surtout liée à l'âge
Edema	0,79	11,0	Presque toujours sur cliché AP (patient alité)
Fibrosis	0,72	0,30	Âge, suivi long, cliché PA
Consolidation	0,71	3,1	Vue AP
Effusion	0,68	1,5	
Cardiomegaly	0,58	1,2	Information dans l'image
Mass	0,53	0,93	Clinique inutile

La clinique n'est donc ni inutile ni suffisante, et son utilité dépend fortement de la pathologie.

La vue change le sens de l'image. Un cliché AP est pris de face, source proche, souvent au lit : le cœur y paraît agrandi par effet de projection, et les patients sont plus graves. Edema est 11 fois plus fréquent en AP qu'en PA, Consolidation 3 fois. C'est exactement la situation où une même caractéristique visuelle (une silhouette cardiaque large, une opacité diffuse) ne doit pas être interprétée de la même manière selon le contexte.

Le contexte peut aussi être un raccourci. Le numéro de suivi vaut 20 en moyenne dans le test contre 5 en train ; la vue AP passe de 34,8 % à 56,6 %. Un modèle qui s'appuie trop sur ces variables apprend « patient grave » plutôt que « épanchement visible ». Zech et al. [21] ont montré ce genre de dérive sur des radiographies issues de plusieurs hôpitaux. L'architecture doit donc laisser la clinique moduler l'image sans pouvoir s'y substituer.

Critère	Ce que montrent nos données	Conséquence
Nature des modalités	Image 1024×1024 (carte 1024×7×7 après DenseNet) contre 4 scalaires. Pas de séquence côté clinique.	Fusion précoce exclue ; TFN, MCB et attention croisée surdimensionnés pour 4 valeurs.
Alignement	Appariement 1 pour 1 par nom de fichier, vérifié à 100 % par le pipeline. Pas de temporalité.	Aucun module d'alignement nécessaire.
Interactions	La vue et l'âge changent la prévalence et l'aspect visuel (Edema ×11 en AP).	Il faut une interaction fine, que la fusion tardive ne permet pas.
Modalités manquantes	0 % en entraînement. En production, la vue ou l'âge peuvent manquer (DICOM incomplet, autre hôpital).	Jeton « clinique absente » appris + suppression aléatoire à l'entraînement.
Volume	77 724 images d'entraînement, 14 classes très déséquilibrées (Hernia : 127 positifs en train).	Encodeur pré-entraîné affiné ; pas assez pour un transformer multimodal entraîné de zéro.

Calcul	GPU Kaggle (T4 ou P100, quota hebdomadaire de 30 h).	DenseNet-121 en 224×224 ; module de fusion de quelques centaines de milliers de paramètres.
Interprétabilité	Contexte médical : il faut savoir si la décision vient de l'image ou du contexte.	Mécanisme dont la contribution se mesure (porte, γ/β), tête auxiliaire image seule.

> **Choix retenu : fusion intermédiaire par conditionnement FiLM de la carte visuelle par la clinique, suivie d'une porte apprise (type GMU) qui dose l'apport direct du vecteur clinique.** FiLM répond au besoin d'interaction (la vue règle la lecture de l'image) pour un coût négligeable ; la porte rend mesurable la part de la clinique ; une perte auxiliaire image seule et la suppression aléatoire de la clinique empêchent le raccourci « vue AP = malade ». L'attention croisée est gardée comme variante dans les ablations, la concaténation et la fusion tardive comme baselines.

5. Architecture et algorithmes proposés

5.1 Schéma d'architecture

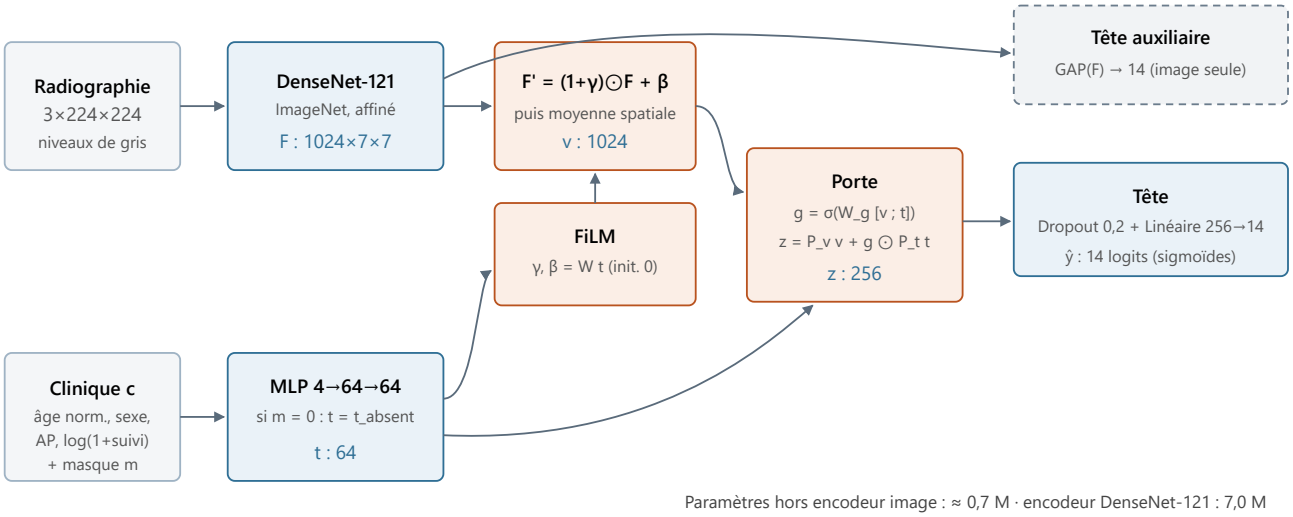


Figure 2 – Architecture retenue (prototype : `src/fusion/model.py`, classe `ChestXrayFusionNet`).

Encodeur image. DenseNet-121 pré-entraîné sur ImageNet, utilisé dans CheXNet [20] et donc éprouvé sur ce jeu ; nous retirons sa tête et gardons la carte 1024×7×7. Il est affiné, avec un taux d'apprentissage dix fois plus faible que le reste. Pour accélérer les premiers essais, la Tâche 4 commencera avec l'encodeur figé et des cartes mises en cache (étape `extract_features`).
Encodeur clinique. MLP 4 → 64 → 64 avec ReLU. L'encodage (âge centré-réduit, sexe, AP, log(1 + suivi)) est le même que dans l'étape `analyze`, pour que les mesures de la section 4 restent comparables.
Fusion. FiLM sur la dernière carte, puis moyenne spatiale, puis porte (d = 256).
Tête. Dropout 0,2 et une couche linéaire vers 14 logits.

5.2 Description formelle du module de fusion

Pour un exemple, x est l'image, $c \in \mathbb{R}^4$ les variables cliniques et $m \in \{0, 1\}$ le masque de présence de la clinique.

$$F = E_{img}(x) \in \mathbb{R}^{C \times H \times W}, \quad C = 1024, H = W = 7$$

$$t = m \cdot E_{tab}(c) + (1 - m) \cdot t_{\emptyset} \in \mathbb{R}^{64} \quad (t_{\emptyset} \text{ appris})$$

$$[\gamma\,;\,\beta] = W_f\,t + b_f,\quad \gamma,\beta \in \mathbb{R}^C,\quad W_f \text{ et } b_f \text{ initialisés à } 0$$

$$F'_{k,i,j} = (1+\gamma_k)\,F_{k,i,j} + \beta_k$$

$$v = (1\,/\,HW)\,\Sigma_{i,j}\,F'_{:,i,j} \in \mathbb{R}^C$$

$$g = \sigma(W_g\,[v\,;\,t] + b_g) \in (0,1)^{256}$$

$$z = P_v\,v + g \odot P_t\,t \in \mathbb{R}^{256}$$

$$\hat{y} = W_h\,z + b_h \in \mathbb{R}^{14},\quad \hat{y}_{img} = W_a\,GAP(F) + b_a$$

Deux détails comptent. On écrit $(1 + \gamma)$ plutôt que γ, et on initialise W_f à zéro : au départ $F' = F$, le réseau se comporte comme un modèle image seule et n'apprend la modulation que si elle fait baisser la perte. Cela évite de détruire les caractéristiques pré-entraînées dans les premières itérations. Ensuite, la porte g ne s'applique qu'au chemin clinique direct : l'image passe toujours. La moyenne de g sur un ensemble de test donne une mesure simple de la part du contexte.

5.3 Pseudo-algorithme

```
Entrées : batch {x, c, m, y} issu de data/processed/dataset.parquet (ChestXrayDataset)
        encodeurs E_img, E_tab ; jeton t_∅ ; module FiLM+porte F ; têtes H, H_aux
        p_drop = 0,2 ; λ = 0,3 ; pos_weight w ∈ R^14

Passe avant
 1. Fmap ← E_img(x)                          # Fmap ∈ R^{B×1024×7×7}
 2. si entraînement : m ← m ⊙ Bernoulli(1 − p_drop) # suppression aléatoire de la clinique
 3. t ← m · E_tab(c) + (1 − m) · t_∅          # t ∈ R^{B×64}
 4. γ, β ← découpe(W_f t + b_f)               # γ, β ∈ R^{B×1024}
 5. v ← moyenne_spatiale((1 + γ) ⊙ Fmap + β)  # v ∈ R^{B×1024}
 6. g ← σ(W_g [v ; t]) ;  z ← P_v v + g ⊙ P_t t   # z ∈ R^{B×256}
 7. ŷ ← H(z) ;  ŷ_img ← H_aux(moyenne_spatiale(Fmap))   # R^{B×14}

Entraînement (une époque)
 8. L ← BCE_w(ŷ, y) + λ · BCE_w(ŷ_img, y)
 9. rétropropagation ; AdamW (lr 1e-4 encodeur image, 1e-3 reste, décroissance cosinus)
10. fin d'époque : AUC moyenne sur val ; sauvegarde du meilleur modèle ;
    journalisation MLflow (paramètres, hash DVC de dataset.parquet, AUC par classe, moyenne de g)
```

5.4 Fonction de perte

$$\mathcal{L} = BCE_w(\hat{y}, y) + \lambda \cdot BCE_w(\hat{y}_{img}, y),\quad \lambda = 0{,}3$$

$$BCE_w(\hat{y}, y) = -(1/14)\,\Sigma_k\,[\,w_k\,y_k \log\sigma(\hat{y}_k) + (1-y_k)\log(1-\sigma(\hat{y}_k))\,],\quad w_k = \sqrt{((N-P_k)\,/\,P_k)}$$

Le poids naïf $(N - P)/P$ vaut 611 pour Hernia et 5,3 pour Infiltration : avec ce rapport, quelques images de hernie dominent le gradient. La racine carrée ramène l'écart à 24,7 contre 2,3, ce qui corrige le déséquilibre sans rendre l'entraînement instable. La perte focale sera testée en alternative. La perte auxiliaire image seule joue le rôle de la perte par modalité recommandée par Wang et al. [19] : la branche image reste obligée de prédire seule, ce qui empêche la clinique de devenir la voie la plus facile. Nous n'ajoutons pas de perte contrastive : nos deux modalités ne décrivent pas la même chose (l'âge n'est pas une « légende » de l'image), l'alignement CLIP n'a pas de sens ici.

5.5 Modalités manquantes et déséquilibre

- **Clinique absente** : le masque m, produit par le pipeline (`has_tabular`), remplace le vecteur par un jeton appris $t_\emptyset$. Le test `test_missing_clinical_data_uses_learned_token` vérifie que les valeurs cliniques sont alors bien ignorées.

- **Suppression aléatoire** (ModDrop [18]) : 20 % des exemples de chaque batch sont entraînés sans clinique. Le modèle reste utilisable si la vue ou l'âge manquent, et ne peut pas compter uniquement sur eux.
- **Image absente** : elle n'arrive pas dans nos données (couverture 100 %, contrôle bloquant). Sans image, on ne pose pas de diagnostic radiologique : ce cas est refusé plutôt que traité.
- **Déséquilibre entre modalités** : initialisation de FiLM à l'identité, tête auxiliaire, suppression aléatoire, et suivi de la moyenne de g au fil des époques.

5.6 Baselines de comparaison

Modèle	Description	Ce qu'il permet de mesurer
B1 – clinique seule	Régression logistique, puis gradient boosting, sur les 4 variables (déjà fait : AUC 0,66)	Plancher : ce que le contexte apporte sans l'image
B2 – image seule	DenseNet-121 + tête linéaire, même entraînement	La référence à battre
B3 – fusion tardive	Moyenne pondérée, puis stacking, des probabilités de B1 et B2	Apport du contexte sans interaction
B4 – concaténation	[GAP(F) ; t] → MLP, bout en bout	Fusion intermédiaire la plus simple
Proposé	FiLM + porte + perte auxiliaire + ModDrop	Apport du conditionnement

5.7 Protocole d'évaluation

Métriques. AUC-ROC par pathologie et moyenne (la métrique de référence sur NIH), AUC-PR pour les classes rares, et calibration (ECE). Résultats rapportés sur le test officiel, avec intervalles de confiance par bootstrap *par patient* (1 000 tirages), puisque les images d'un même patient ne sont pas indépendantes. Comme vu en section 2.5, l'écart val/test sera rapporté aussi.

Sous-groupes. AUC par sexe, tranche d'âge et vue. Seyyed-Kalantari et al. [22] ont mesuré des taux de sous-diagnostic plus élevés chez certains groupes avec des modèles entraînés sur ce jeu ; nous voulons vérifier que la fusion n'aggrave pas ces écarts.

Ablations.

1. Retrait d'une modalité à l'inférence (m = 0 partout) : la perte de performance mesure la dépendance au contexte.
2. Permutation de la clinique entre exemples du batch : si l'AUC ne bouge pas, la clinique ne sert à rien ; si elle s'effondre, le modèle en dépend trop.
3. Remplacement du module : FiLM seul, porte seule, concaténation, attention croisée (requête clinique sur les 49 positions de la carte 7×7).
4. Sans `followup` : vérifier que le gain ne vient pas du raccourci « patient très suivi ».
5. Sans perte auxiliaire, sans ModDrop, $\lambda \in \{0 ; 0{,}3 ; 1\}$.

5.8 Prototype (bonus)

Le module complet est implémenté dans `src/fusion/model.py`, et un Dataset PyTorch lit la table du pipeline dans `src/fusion/dataset.py`. Le fichier `tests/test_fusion.py` charge un vrai batch de 4 radiographies de l'échantillon et vérifie :

- les dimensions du batch (4×3×224×224, 4×4, 4×14) ;
- la passe avant : logits 4×14, logits auxiliaires 4×14, porte 4×256 ;
- la perte finie et un gradient non nul jusqu'aux poids de FiLM ;
- l'invariance aux valeurs cliniques quand m = 0 ;
- $\gamma = \beta = 0$ à l'initialisation, et une porte dans (0, 1).

Les quatre tests passent en local et dans la CI (encodeur non pré-entraîné pour ne pas télécharger les poids à chaque exécution).

6. Plan d'intégration et prochaines étapes

Étape	Changement dans le pipeline	Tâche
`extract_features`	Nouvelle étape DVC après `validate` : passe DenseNet-121 figé sur toutes les images, met en cache les cartes (ou leur moyenne) dans `data/features/`. Permet d'itérer sur le module de fusion sans recalculer l'encodeur.	4
Prétraitement image	Conversion RGBA → L (2 cas sur 300 contrôlés), redimensionnement 224, normalisation ImageNet, augmentation légère en train (rotation ±7°, translation). Paramètres dans la section `fusion`.	4
`train`	Baselines B1 à B4 puis modèle proposé ; un run MLflow par configuration ; le hash DVC de `dataset.parquet` est enregistré dans chaque run.	4
`evaluate`	Métriques, sous-groupes et ablations ; `reports/eval.json` déclaré en métrique DVC pour `dvc metrics diff` entre versions.	4-5
Contrôles	Nouveaux contrôles : intégrité des caractéristiques en cache (dimensions, NaN), même ensemble d'images que `dataset.parquet`.	4
Explicabilité	Grad-CAM sur la carte modulée F', analyse de γ par vue (que module la vue AP ?).	5

Ce que nous retenons de cette tâche : le pipeline a déjà rendu service. Sans le contrôle d'écart de prévalence, nous aurions sans doute comparé nos modèles sur une validation qui ne ressemble pas au test, et sans l'étape d'analyse nous aurions probablement choisi l'attention croisée parce qu'elle est à la mode, alors que nos données cliniques tiennent en quatre nombres.

Références

1. T. Baltrušaitis, C. Ahuja, L.-P. Morency, « Multimodal Machine Learning: A Survey and Taxonomy », *IEEE TPAMI*, 41(2), 2019.
2. A. Zadeh, M. Chen, S. Poria, E. Cambria, L.-P. Morency, « Tensor Fusion Network for Multimodal Sentiment Analysis », *EMNLP*, 2017.
3. Z. Liu, Y. Shen, V. B. Lakshminarasimhan, P. P. Liang, A. Zadeh, L.-P. Morency, « Efficient Low-rank Multimodal Fusion with Modality-Specific Factors », *ACL*, 2018.
4. J. Arevalo, T. Solorio, M. Montes-y-Gómez, F. A. González, « Gated Multimodal Units for Information Fusion », *ICLR Workshop*, 2017.
5. E. Perez, F. Strub, H. de Vries, V. Dumoulin, A. Courville, « FiLM: Visual Reasoning with a General Conditioning Layer », *AAAI*, 2018.
6. Y.-H. H. Tsai, S. Bai, P. P. Liang, J. Z. Kolter, L.-P. Morency, R. Salakhutdinov, « Multimodal Transformer for Unaligned Multimodal Language Sequences », *ACL*, 2019.
7. A. Nagrani, S. Yang, A. Arnab, A. Jansen, C. Schmid, C. Sun, « Attention Bottlenecks for Multimodal Fusion », *NeurIPS*, 2021.
8. A. Radford et al., « Learning Transferable Visual Models From Natural Language Supervision », *ICML*, 2021.
9. D. Kreuzberger, N. Kühl, S. Hirschl, « Machine Learning Operations (MLOps): Overview, Definition, and Architecture », *IEEE Access*, 11, 2023.
10. Documentation officielle : DVC (dvc.org/doc), pandera (pandera.readthedocs.io), GitHub Actions (docs.github.com/actions).
11. X. Wang, Y. Peng, L. Lu, Z. Lu, M. Bagheri, R. M. Summers, « ChestX-ray8: Hospital-scale Chest X-ray Database and Benchmarks on Weakly-Supervised Classification and Localization of Common Thorax Diseases », *CVPR*, 2017.
12. D. S. Kermany et al., « Identifying Medical Diagnoses and Treatable Diseases by Image-Based Deep Learning », *Cell*, 172(5), 2018.
13. S.-C. Huang, A. Pareek, S. Seyyedi, I. Banerjee, M. P. Lungren, « Fusion of medical imaging and electronic health records using deep learning: a systematic review and implementation guidelines », *npj Digital Medicine*, 3, 2020.

14. A. Fukui, D. H. Park, D. Yang, A. Rohrbach, T. Darrell, M. Rohrbach, « Multimodal Compact Bilinear Pooling for Visual Question Answering and Visual Grounding », *EMNLP*, 2016.
15. Z. Yu, J. Yu, J. Fan, D. Tao, « Multi-modal Factorized Bilinear Pooling with Co-Attention Learning for Visual Question Answering », *ICCV*, 2017.
16. A. Vaswani et al., « Attention Is All You Need », *NeurIPS*, 2017.
17. T. N. Kipf, M. Welling, « Semi-Supervised Classification with Graph Convolutional Networks », *ICLR*, 2017.
18. N. Neverova, C. Wolf, G. Taylor, F. Nebout, « ModDrop: Adaptive Multi-Modal Gesture Recognition », *IEEE TPAMI*, 38(8), 2016.
19. W. Wang, D. Tran, M. Feiszli, « What Makes Training Multi-Modal Classification Networks Hard? », *CVPR*, 2020.
20. P. Rajpurkar et al., « CheXNet: Radiologist-Level Pneumonia Detection on Chest X-Rays with Deep Learning », arXiv:1711.05225, 2017.
21. J. R. Zech, M. A. Badgeley, M. Liu, A. B. Costa, J. J. Titano, E. K. Oermann, « Variable generalization performance of a deep learning model to detect pneumonia in chest radiographs: A cross-sectional study », *PLOS Medicine*, 15(11), 2018.
22. L. Seyyed-Kalantari, H. Zhang, M. B. A. McDermott, I. Y. Chen, M. Ghassemi, « Underdiagnosis bias of artificial intelligence algorithms applied to chest radiographs in under-served patient populations », *Nature Medicine*, 27, 2021.
23. L. Oakden-Rayner, « Exploring Large-scale Public Medical Image Datasets », *Academic Radiology*, 27(1), 2020.